

PETSc to MATLAB Solution Map

slope_stability

A chapter-by-chapter map from the original MATLAB workflow to the PETSc-based Python implementation

Repository: **slope_stability** checkout

Audience: original MATLAB authors and maintainers

Intent: fastest path to locating each old concept in the PETSc codebase

Date: April 30, 2026

Reading Guide

This document is split into two parts.

- **Mainline path** follows only the committed default 3D benchmark in `benchmarks/slope_stability_3D_hetero_SSR_default/case.toml`.
- **Annex** collects code paths that are present in the repository but inactive in that benchmark: seepage, alternative constitutive ownership modes, legacy tangent kernels, BDDC, Hypre/GAMG backends, direct SSR, limit load, and other optional continuation features.

Each chapter pair answers four questions:

- what the MATLAB code did in that part of the workflow,
- where the default PETSc responsibility now lives,
- what changed on the current parallel PMG path,
- where to look next if the inactive or alternative code paths matter.

The reference thread throughout the mainline part is the classical 3D heterogeneous SSR case, centered on `slope_stability_matlab/slope_stability_3D_hetero_SSR.m` and the config-driven PETSc entrypoint `src/slope_stability/cli/run_case_from_config.py`. Internally, `src/slope_stability/execution/asset_case/runner.py` dispatches the resolved asset to the 3D mechanics route, with `constitutive_mode=overlap`, `tangent_kernel=rows`, `method=indirect`, and `pc_backend=pmg_shell`.

Contents

I	Mainline Path	1
1	Execution Map	2
1.1	Benchmark Definition	2
1.2	MATLAB Driver to PETSc Runner	2
1.3	Mainline Call Chain	3
1.4	What Moved Out of the MATLAB Driver	3
1.5	Shortest Reading Order	4
1.6	Where To Look For Inactive Paths	4
2	Mesh, Materials, and Load Assembly	5
2.1	Mainline Preprocessing Sequence	5
2.2	Object Mapping on the Mainline Path	5
2.3	Node Reordering Is Part of The Mainline	6
2.4	Material Expansion Is Still A Direct Translation	6
2.5	What The Mainline Path Does Not Build	6
2.6	Where To Edit	6
2.7	Where The Other Preprocessing Paths Went	7
3	Seepage Coupling	8
3.1	What The Mainline Mechanical Path Uses Instead	8
3.2	Where The Branch Would Enter If Enabled	8
4	Constitutive Relationships and Reduction	9
4.1	Mainline Constitutive Cycle	9
4.2	What The Mainline Builder Owns	9
4.3	Mainline Ownership Mode: Overlap	9
4.4	Which Builder Interfaces Are Mainline	10
4.5	What Stayed The Same For MATLAB Authors	10
4.6	Where To Edit	10
4.7	Where The Other Constitutive Modes Went	10
5	Tangent Assembly	11
5.1	Mainline Path In One Line	11

5.2	Why This Path Is Selected	11
5.3	Step 1: Owned Elastic Rows	11
5.4	Step 2: Fixed Owned Tangent Pattern	12
5.5	Step 3: Overlap Constitutive Evaluation	12
5.6	Step 4: Row-Slot Tangent Kernel	12
5.7	Step 5: Internal Force And Tangent Refresh	13
5.8	Step 6: Regularization On The Same Pattern	13
5.9	Constrained Rows Stay Explicit	13
5.10	What The PMG Shell Solver Receives	13
5.11	Where To Edit	14
5.12	Where The Other Tangent Paths Went	14
6	Linear Solvers and PETSc	15
6.1	Mainline Role Map	15
6.2	What The Mainline Solver Actually Means	15
6.3	PMG Shell Requirements	16
6.4	Mainline Solver Lifecycle	16
6.5	How The Mainline Path Relates To PMG	16
6.6	Where To Edit	16
6.7	Where The Other Solver Families Went	16
7	Newton Solvers and Damping	17
7.1	Current Mainline Newton Branch	17
7.2	Plain Newton On The Mainline Path	17
7.3	Nested SSR Solve On The Mainline Path	18
7.4	Stopping And Damping In The Current Benchmark	18
7.5	What Newton Receives From The Constitutive Layer	18
7.6	Deflation Basis On The Mainline Path	18
7.7	Where To Edit	18
7.8	Where The Inactive Newton Branches Went	19
8	Continuation Algorithms	20
8.1	Mainline Continuation Contract	20
8.2	Mainline Call Chain	20
8.3	Accepted-Step Lifecycle	21
8.4	What The Legacy Controller Means Here	21
8.5	Mainline History Output	21
8.6	Where To Edit	21
8.7	Where The Other Continuation Features Went	21

II	Annex: Inactive and Alternative Paths	22
A	Execution Map: Inactive and Alternative Entry Paths	23
A.1	Generic Framework Defaults	23
A.2	Other Runner Surfaces	23
A.3	Why The Split Matters	24
B	Mesh, Materials, and Load Assembly: Inactive Paths	25
B.1	Global Reference Assembly	25
B.2	Asset-First Mesh Loading	25
B.3	Alternative Node Orderings	25
B.4	Why The Global Path Remains Important	26
C	Seepage Coupling: Inactive Branch	27
C.1	Exact Seepage-to-Mechanics Sequence	27
C.2	What The PETSc Split Changed	27
C.3	Main Edit Points	28
D	Constitutive Relationships and Reduction: Inactive Modes	29
D.1	Alternative Ownership Modes	29
D.2	Why They Are Not Mainline Today	29
D.3	Additional Builder Interfaces	29
E	Tangent Assembly: Inactive and Alternative Paths	30
E.1	Global MATLAB-Style Reference Path	30
E.2	Unique-Owner Constitutive Recovery	30
E.3	Legacy Scatter Kernel	30
E.4	BDDC Local-Subdomain Pattern	30
E.5	Why The Main Text Excludes Them	31
F	Linear Solvers and PETSc: Inactive Backends and Policies	32
F.1	Other Solver Families	32
F.2	Other Backends	32
F.3	Why BDDC Is Separate	32
F.4	Other Explicit Policy Controls	32
G	Newton Solvers and Damping: Inactive Branches	34
G.1	Full-System Operator Mode	34
G.2	Local Full-Operator Builders	34
G.3	Separate Explicit Preconditioning Matrices	34
G.4	Optional Warm Starts	34
H	Continuation Algorithms: Inactive Modes and Extensions	35
H.1	Direct SSR	35

H.2	Limit Load	35
H.3	Predictor Families Beyond Secant	35
H.4	Additional Control-Flow Extensions	35
H.5	Why They Were Moved Out Of The Main Text	36

Part I

Mainline Path

Execution Map

This chapter follows only the committed default benchmark `benchmarks/slope_stability_3D_hetero_SSR_default/case.toml`. In that benchmark the main path is:

3D hetero SSR → indirect continuation → owned-row assembly → PMG shell backend.

1.1 Benchmark Definition

For the current mainline reading path, the important settings are:

- `analysis=ssr`,
- `asset=3d_hetero_slope`,
- `mesh_variant=adaptive_family_a_l1.msh`,
- `profile=default`,
- `method=indirect`,
- `elem_type=P4`,
- `node_ordering=blockmetis`,
- `mpi_distribute_by_nodes=true`,
- `constitutive_mode=overlap`,
- `tangent_kernel=rows` by execution default,
- `solver_type=PETSC_MATLAB_DFGMRES_HYPRE_NULLSPACE`,
- `pc_backend=pmg_shell`.

Everything else in this document is organized around that exact path. Generic framework defaults and other runner surfaces are moved to [Appendix A](#).

1.2 MATLAB Driver to PETSc Runner

The MATLAB reference script `slope_stability_matlab/slope_stability_3D_hetero_SSR.m` concentrated the whole solve in one file. The current PETSc path is the same algorithm broken into explicit stages:

1. `src/slope_stability/cli/run_case_from_config.py` receives the case config.
2. `src/slope_stability/core/run_config.py` loads the TOML, derives dimension from the asset, and rejects legacy problem-owned fields such as raw mesh paths.
3. `src/slope_stability/problem_asset_runtime.py` resolves the asset, mesh variant, and profile.
4. `src/slope_stability/execution/asset_case/runner.py` selects the generic 3D mechanics route.

5. `src/slope_stability/execution/asset_case/mechanics_3d.py::run_capture` performs mesh construction, constitutive-builder setup, solver construction, and continuation launch.
6. `src/slope_stability/continuation/indirect.py::SSR_indirect_continuation` drives the branch tracking.
7. `src/slope_stability/nonlinear/newton.py::newton_ind_ssr` performs the nested Newton solves used at each continuation step.
8. `src/slope_stability/linear/solver.py::SolverFactory.create` provides the PMG-shell preconditioned outer Krylov solve.

1.3 Mainline Call Chain

For the default 3D PMG benchmark, the runner sequence is:

1. resolve asset `3d_hetero_slope` and mesh variant `adaptive_family_a_l1.msh`,
2. build the promoted solver mesh from the canonical asset definition,
3. reorder nodes with `reorder_mesh_nodes(...)` using `blockmetis`,
4. expand material rows with `heterogenous_materials(...)`,
5. choose the owned distributed assembly path through `src/slope_stability/cli/assembly_policy.py`,
6. build owned elastic rows with `assemble_owned_elastic_rows_for_comm(...)`,
7. create `ConstitutiveOperator`,
8. prepare the owned tangent pattern with `prepare_owned_tangent_pattern(...)`,
9. create the linear solver through `SolverFactory.create(...)`,
10. launch `SSR_indirect_continuation(...)`.

That list is the modern replacement for reading the MATLAB script from top to bottom.

1.4 What Moved Out of the MATLAB Driver

The main structural change is not a new nonlinear algorithm. It is that the old driver responsibilities are now explicit modules:

MATLAB role	MATLAB location	Mainline PETSc location
Case declaration	top of <code>slope_stability_matlab/slope_stability_3D_hetero_SSR.m</code>	<code>case.toml</code> plus <code>src/slope_stability/core/run_config.py</code>
Mesh and material preprocessing	script body plus <code>slope_stability_matlab/+MESH/*</code> and <code>slope_stability_matlab/+ASSEMBLY/*</code>	<code>src/slope_stability/problem_assemble_runtime.py</code> , <code>src/slope_stability/execution/asset_case/mechanics_3d.py</code> , <code>src/slope_stability/mesh/*</code> , <code>src/slope_stability/fem/*</code>
Constitutive handle construction	<code>slope_stability_matlab/+CONSTITUTIVE_PROBLEM/CONSTITUTIVE.m</code>	<code>src/slope_stability/constitutive/problem.py::ConstitutiveOperator</code>
Linear solver setup	<code>slope_stability_matlab/+LINEAR_SOLVERS/set_linear_solver.m</code>	<code>src/slope_stability/linear/solver.py::SolverFactory.create</code>

MATLAB role	MATLAB location	Mainline PETSc location
Continuation launch	<code>slope_stability_matlab/+C0 NTINUATION/SSR_indirect_co ntinuation.m</code>	<code>src/slope_stability/continuation/indirect.py::SSR_indirect_c ontinuation</code>

1.5 Shortest Reading Order

For the current mainline path, the shortest route through the PETSc code is:

1. `src/slope_stability/cli/run_case_from_config.py` and `src/slope_stability/execution/asset_case/runner.py` for orchestration,
2. `src/slope_stability/execution/asset_case/mechanics_3d.py` for 3D mechanics setup,
3. `src/slope_stability/cli/assembly_policy.py` for the owned-path choice,
4. `src/slope_stability/fem/distributed_elastic.py` and `src/slope_stability/fem/distributed_tangent.py` for matrix construction,
5. `src/slope_stability/constitutive/problem.py` for the constitutive builder,
6. `src/slope_stability/linear/solver.py` for the PMG-shell solver path,
7. `src/slope_stability/nonlinear/newton.py` and `src/slope_stability/continuation/indirect.py` for branch advancement.

1.6 Where To Look For Inactive Paths

This chapter intentionally excludes generic framework defaults, seepage runners, and other entry points that are not used by the default parallel PMG benchmark. Those are collected in [Appendix A](#).

Mesh, Materials, and Load Assembly

On the current parallel PMG path, preprocessing still serves the same purpose as in MATLAB: turn the mesh and material table into the objects needed by the constitutive builder and continuation solver. The important difference is that the default path no longer starts by building one global **B** matrix and one global elastic matrix.

2.1 Mainline Preprocessing Sequence

For the committed 3D benchmark, `src/slope_stability/execution/asset_case/mechanics_3d.py` receives the asset-resolved mesh and problem data through `src/slope_stability/execution/asset_case/runner.py`. The sequence is:

1. resolve `3d_hetero_slope` and `adaptive_family_a_l1.msh` from `case.toml`,
2. build the canonical Gmsh mesh through the asset API,
3. reorder nodes with `block_metis`,
4. compute `n_q` from `quadrature_volume_3d(elem_type)`,
5. expand material rows over integration points through `heterogenous_materials(...)`,
6. choose the lightweight MPI elastic path,
7. assemble owned elastic rows through `assemble_owned_elastic_rows_for_comm(...)`,
8. convert those rows into a PETSc AIJ matrix `K_elast`,
9. gather the local right-hand-side pieces into the global body-force field `f_v`.

So the current default path reaches the same physical objects as MATLAB, but it reaches them through rank-owned assembly rather than a global sparse build.

2.2 Object Mapping on the Mainline Path

Object	MATLAB meaning	Mainline PETSc meaning
<code>coord</code>	nodal coordinates	reordered coordinates aligned with PETSc ownership
<code>elem</code>	element connectivity	same connectivity after node permutation
<code>surf</code>	boundary faces	same boundary meaning after node permutation
<code>q_mask</code>	free displacement mask	same mask, reused later by solver and free-DOF extraction
<code>material_identifier</code>	material row id per element	same element-wise id after reorder

Object	MATLAB meaning	Mainline PETSc meaning
<code>c0, phi, psi, shear, bulk, lame, gamma</code>	integration-point material arrays	same arrays built by <code>heterogenous_materials(...)</code>
<code>K_elast</code>	elastic stiffness matrix	PETSc AIJ matrix assembled from owned rows
<code>f_V</code>	body-force vector	global field formed by gathering rank-local body-force pieces

2.3 Node Reordering Is Part of The Mainline

For the current benchmark, node reordering is not an optional PETSc tuning detail. It is part of the default path because the runner sets `node_ordering=block_metis`. That means:

- the numbering is chosen to align with MPI ownership,
- later owned-row matrix assembly uses that numbering directly,
- the constitutive and tangent chapters inherit this reordered ownership map.

MATLAB did not need an explicit public reordering stage because its global matrix lived in one address space.

2.4 Material Expansion Is Still A Direct Translation

`src/slope_stability/mesh/materials.py::heterogenous_materials` is still the closest translation of `slope_stability_matlab/+ASSEMBLY/heterogenous_materials.m`. It still:

- interprets material ids element-wise,
- converts `phi` and `psi` to radians,
- derives `shear`, `bulk`, and `lame`,
- repeats material rows over the element quadrature rule,
- builds the integration-point weight term `gamma`.

For the non-seepage mainline run, the saturation mask is effectively fixed, so the mechanical body-force path is determined directly from the material table without any preceding seepage solve.

2.5 What The Mainline Path Does Not Build

The most important difference from the earlier document version is this:

- the current default PMG path does **not** use the global `assemble_strain_operator(...)+build_elastic_stiffness_matrix(...)` route as its normal startup path,
- instead it builds owned elastic rows immediately and lets the owned tangent pattern carry the detailed strain geometry for later Newton updates.

The global reference assembly is still available, but it belongs in the annex because it is not the default parallel PMG route.

2.6 Where To Edit

- `src/slope_stability/mesh/loader.py` for mesh loading,

- `src/slope_stability/mesh/reorder.py` for ownership-oriented numbering,
- `src/slope_stability/mesh/materials.py` for integration-point material expansion,
- `src/slope_stability/fem/distributed_elastic.py` for the owned elastic precursor,
- `src/slope_stability/execution/asset_case/mechanics_3d.py` for the exact mainline assembly order.

2.7 Where The Other Preprocessing Paths Went

The global reference assembly, other node orderings, and seepage-oriented mesh loaders are moved to Appendix [B](#).

Seepage Coupling

For the committed default benchmark, this chapter is intentionally short because the seepage branch is **inactive**. The case config selects `analysis=ssr` on asset `3d_hetero_slope`, which has no seepage capability, so the mainline 3D PMG solve does not run any hydraulic preprocessing or pore-pressure Newton loop.

3.1 What The Mainline Mechanical Path Uses Instead

On the default path:

- material properties come directly from the mechanical material table,
- the body-force field comes directly from the mechanical `gamma`-based volume load,
- no pore-pressure field `pw` is solved,
- no pressure gradient `grad_p` is transferred into the mechanical forcing,
- no saturation update changes the material state before the mechanical solve.

This is exactly why the current benchmark can go straight from mesh/material preprocessing to constitutive setup and SSR continuation.

3.2 Where The Branch Would Enter If Enabled

If seepage were enabled, it would branch *before* the mechanical load and constitutive setup:

mesh and material load → seepage solve
→ (`pw`, `grad_p`, `mater_sat`)
→ mechanical load and constitutive state.

That whole path is documented in Appendix C. It is real code, but it is not part of the default parallel PMG run.

Constitutive Relationships and Reduction

The constitutive law itself is unchanged on the current PMG path:

- the same Mohr-Coulomb return mapping,
- the same Davis reduction,
- the same stress tensor S ,
- the same consistent tangent DS .

What changes on the mainline path is how those quantities are owned, cached, and exposed to distributed PETSc operators.

4.1 Mainline Constitutive Cycle

For the committed benchmark, one constitutive evaluation still means:

1. reduce strength parameters for the current λ ,
2. compute strain from the current displacement field,
3. evaluate stresses and tangent blocks at integration points,
4. assemble the internal force,
5. build or refresh the tangent and regularized operators needed by Newton.

That is the same cycle MATLAB authors already know.

4.2 What The Mainline Builder Owns

The mainline `ConstitutiveOperator` in `src/slope_stability/constitutive/problem.py` owns:

- the material arrays `c0`, `phi`, `psi`, `shear`, `bulk`, `lame`,
- the free-DOF mask `q_mask`,
- the current stress and tangent state,
- the owned tangent metadata supplied by `OwnedTangentPattern`,
- cached PETSc matrices for tangent and regularized operators on that owned pattern.

The widened class boundary is a PETSc requirement, not a constitutive change.

4.3 Mainline Ownership Mode: Overlap

For the committed 3D benchmark, the important constitutive fact is simple:

- `constitutive_mode=overlap`,
- the owned tangent pattern is prepared with `include_unique=false`,
- each rank evaluates constitutive data directly on the overlap integration points it needs for its owned rows,
- no unique-owner gather or exchange stage participates in the mainline run.

So the current benchmark should be read as an *overlap-evaluated constitutive path*, not as a unique-owner recovery path.

4.4 Which Builder Interfaces Are Mainline

Because the linear solver backend is `pmg_shell` and does not prefer the full-system operator, the mainline Newton path mainly consumes:

- `build_F_free(...)` or equivalent free-output residual builders,
- `build_F_K_tangent_reduced_free(...)` and related free-output tangent builders,
- `build_K_regularized(...)` for the regularized owned-row PETSc matrix.

So although the class exposes many interfaces, the default 3D PMG route is the free-output interface family backed by owned-row caches.

4.5 What Stayed The Same For MATLAB Authors

The most important parity statement is still:

- `reduction(lambda)` still means strength reduction,
- strain still comes from the current displacement field,
- `constitutive_problem_3D(...)` still computes the same stresses and tangent blocks,
- the internal force and tangent still represent the same nonlinear mechanics.

The PETSc scaffolding around those steps exists because MPI ownership and solver reuse are now explicit.

4.6 Where To Edit

- `src/slope_stability/constitutive/reduction.py` for Davis reduction rules,
- `src/slope_stability/constitutive/problem.py` for stress, tangent, residual, and regularized-operator exposure,
- `src/slope_stability/fem/distributed_tangent.py` when the owned-row constitutive interface changes.

4.7 Where The Other Constitutive Modes Went

The global and unique-owner constitutive modes, along with the BDDC-specific builder interfaces, are moved to Appendix D.

Tangent Assembly

This is the chapter where the PETSc redesign differs most strongly from the old MATLAB implementation. The mathematical operator is still

$$K_{\text{tangent}} = B^T D_S B,$$

but the default 3D PMG path no longer materializes that object as one globally rebuilt sparse matrix.

5.1 Mainline Path In One Line

For the committed benchmark, the normal tangent route is:

owned elastic rows → owned tangent pattern
→ overlap constitutive data
→ rows kernel
→ regularized free operator for PMG shell.

That is the single path MATLAB authors should hold in mind first.

5.2 Why This Path Is Selected

`src/slope_stability/cli/assembly_policy.py` selects the owned distributed path because:

- `mpi_distribute_by_nodes=true`,
- the solver is not `DIRECT`,
- `constitutive_mode=overlap`.

As a result:

- the runner enables the owned tangent path,
- the runner enables the lightweight owned elastic path,
- the global MATLAB-style tangent rebuild is not used in the normal solve.

5.3 Step 1: Owned Elastic Rows

The current path begins with `assemble_owned_elastic_rows_for_comm(...)` in `src/slope_stability/fem/distributed_elastic.py`. This builds a PETSc-ready elastic precursor by rank-owned rows.

What it stores. The owned elastic object stores:

- the node interval owned by the rank,

- the overlap elements needed to assemble those rows,
- a local CSR matrix containing only the rank-owned global rows,
- the matching local right-hand-side contribution.

Why it matters. On the mainline path, the elastic matrix is not just a one-time startup object. It becomes the structural template for later tangent regularization and for the owned-row sparsity pattern.

5.4 Step 2: Fixed Owned Tangent Pattern

The core PETSc object is `OwnedTangentPattern` in `src/slope_stability/fem/distributed_tangent.py`. For the current benchmark it is prepared with:

- `include_unique=false`,
- `include_legacy_scatter=false`,
- `include_overlap_B=false`.

Those three booleans are the clearest summary of what the current path is *not* doing.

What the pattern precomputes. For the mainline run, the pattern stores:

- the owned row interval and owned free-row mask,
- the overlap node and element patch needed for those rows,
- the fixed CSR sparsity pattern for those rows,
- elastic values projected onto that same pattern,
- overlap strain geometry data needed to re-evaluate the tangent each Newton step,
- constrained diagonal positions for Dirichlet rows.

This is the central architectural difference from MATLAB: the sparse pattern is prepared once, and only the values are refreshed during Newton.

5.5 Step 3: Overlap Constitutive Evaluation

Because the mainline constitutive mode is `overlap`, each rank evaluates stress and tangent data directly on the overlap integration points needed by its owned rows. In practical terms:

- there is no unique-owner recovery stage,
- there is no allgather of constitutive data,
- the tangent kernel receives local overlap data directly.

That is why the overlap mode and the owned-row tangent pattern fit together so well on the PMG path.

5.6 Step 4: Row-Slot Tangent Kernel

The mainline tangent kernel is `rows`. This is the normal production path for the committed benchmark because `tangent_kernel=rows` by execution default and the benchmark does not override it.

How the rows kernel thinks. Instead of building a whole elemental matrix and scattering it afterward, the kernel walks the fixed owned CSR pattern row by row. Its metadata is organized around:

- `row_slot_ptr`,
- `slot_elem`,
- `slot_lrow`,
- `slot_pos`.

So the assembly order is no longer “form k_e , then scatter.” It is “visit this owned sparse row, accumulate exactly the entries needed for this row.” The bilinear form is unchanged. The dataflow is row-oriented instead of element-oriented.

5.7 Step 5: Internal Force And Tangent Refresh

At each Newton iteration the mainline path performs two distributed refreshes:

- `assemble_owned_force_from_local_stress(...)` updates the owned internal-force rows,
- `assemble_owned_tangent_values(...)` refreshes the tangent values on the fixed owned CSR pattern.

This is the PETSc replacement for rebuilding the global $B' * D_p * B$ matrix in MATLAB. The matrix structure stays fixed. Only the values derived from the current constitutive state change.

5.8 Step 6: Regularization On The Same Pattern

The regularized Newton matrix is still

$$K_r = rK_{\text{elast}} + (1 - r)K_{\text{tangent}}.$$

On the mainline path, that blend happens directly on the owned pattern through `assemble_owned_regularized_matrix(...)`:

- the elastic contribution comes from stored `elastic_values`,
- the tangent contribution comes from the current owned tangent refresh,
- the result keeps the same owned CSR structure.

This is why the elastic precursor is carried forward explicitly into the nonlinear solve.

5.9 Constrained Rows Stay Explicit

Dirichlet rows are enforced through stored `constrained_diag_positions`. After each tangent or regularized-matrix refresh, those positions are reset to 1. So constrained rows remain valid even though the unconstrained entries are overwritten every Newton step.

5.10 What The PMG Shell Solver Receives

The current PMG benchmark does not consume the whole operator family exposed by the constitutive layer. The practical flow is:

1. owned elastic rows and owned tangent rows are built in PETSc format,
2. the regularized operator is formed on that owned-row pattern,
3. the free-space operator is extracted for the Krylov solve,

4. the PMG shell backend uses that reduced operator as its fine-grid system.

So the default path is a distributed owned-row assembly underneath a reduced free-space Newton solve.

5.11 Where To Edit

- `src/slope_stability/cli/assembly_policy.py` for path selection,
- `src/slope_stability/fem/distributed_elastic.py` for the elastic precursor,
- `src/slope_stability/fem/distributed_tangent.py` for the owned pattern and the `rows` kernel,
- `src/slope_stability/constitutive/problem.py` for how the cached PETSc matrices are exposed to Newton.

5.12 Where The Other Tangent Paths Went

Legacy scatter assembly, the global reference rebuild, unique-owner constitutive recovery, and BDDC local-subdomain matrices are moved to Appendix [E](#).

Linear Solvers and PETSc

For the committed 3D benchmark, the key linear-solver fact is that the outer solver family and the inner preconditioner backend are now separate controls. The current mainline combination is:

- `solver_type=PETSC_MATLAB_DFGMRES_HYPRE_NULLSPACE`,
- `pc_backend=pmg_shell`.

So the historical solver-type name is preserved, but the actual backend used by the default benchmark is the PMG shell path.

6.1 Mainline Role Map

MATLAB role	MATLAB location	Mainline PETSc location
Solver factory	<code>slope_stability_matlab/+ LINEAR_SOLVERS/set_linear_solver.m</code>	<code>src/slope_stability/linear/solver.py::SolverFactory.create</code>
Deflated FGMRES logic	<code>slope_stability_matlab/+ LINEAR_SOLVERS/DFGMRES.m</code>	MATLAB-style PETSc wrappers in <code>src/slope_stability/linear/solver.py</code>
Iteration logging	<code>slope_stability_matlab/+ LINEAR_SOLVERS/IterationCollector.m</code>	<code>src/slope_stability/linear/collector.py</code>
Near-nullspace helpers	<code>slope_stability_matlab/+ LINEAR_SOLVERS/near_null_space_elasticity_3D.m</code>	<code>src/slope_stability/linear/preconditioners.py</code> and solver setup

6.2 What The Mainline Solver Actually Means

For the default parallel PMG benchmark, the solver stack should be read as:

- **outer solver family:** PETSc-wrapped MATLAB-style deflated FGMRES,
- **fine-grid operator:** the reduced free-space regularized operator coming from the owned-row tangent path,
- **preconditioner backend:** `pmg_shell`.

That is the decisive interpretation. The benchmark is not currently using the Hypre backend despite the historical text inside `solver_type`.

6.3 PMG Shell Requirements

The PMG-shell path enforces a narrower contract than the broader solver framework:

- `preconditioner_matrix_source=tangent`,
- `preconditioner_matrix_policy=current`,
- `preconditioner_rebuild_policy=every_newton`,
- `full_system_preconditioner=false`.

Those requirements matter because they determine which constitutive and Newton interfaces are actually exercised. In particular, they keep the mainline path on the reduced free-space operator rather than the full-system or BDDC branches.

6.4 Mainline Solver Lifecycle

At each nonlinear step, the solver side does the following:

1. receive the current reduced operator from Newton,
2. decide whether the PMG preconditioner should be rebuilt,
3. configure the PMG shell multilevel objects,
4. solve the reduced linear system with the PETSc/MATLAB deflated FGMRES wrapper,
5. record iteration counts, solve time, preconditioner time, and orthogonalization time.

This is the same broad job as the MATLAB solver object, but PETSc makes the preconditioner lifecycle explicit.

6.5 How The Mainline Path Relates To PMG

The custom `pmg_shell` backend is the current default PMG route for the 3D benchmark. In practical terms it means:

- the fine operator comes from the distributed owned-row tangent assembly,
- the operator passed to the solve is the reduced free-space operator,
- the preconditioner is rebuilt from the current tangent each Newton step.

That is the mainline meaning of “parallel PMG” in this repository today.

6.6 Where To Edit

- `src/slope_stability/linear/solver.py` for solver-family selection, PMG-shell setup, and preconditioner lifecycle,
- `src/slope_stability/linear/preconditioners.py` for helper objects and nullspace construction,
- `src/slope_stability/linear/collector.py` for iteration and timing collection.

6.7 Where The Other Solver Families Went

Hypr, GAMG, BDDC, direct solves, built-in PETSc `pmg`, and other solver policies are moved to [Appendix F](#).

Newton Solvers and Damping

The default 3D PMG benchmark uses the same nonlinear idea as MATLAB:

- build residual and tangent information,
- regularize the tangent,
- solve for a correction,
- line-search the update,
- accept or reject the iteration.

What changed is that the current path is now explicitly tied to the reduced free-space operator produced by owned-row distributed assembly.

7.1 Current Mainline Newton Branch

For the committed benchmark, the Newton decision tree collapses to one branch:

- the backend is `pmg_shell`,
- `pmg_shell` enforces `full_system_preconditioner=false`,
- Newton therefore uses the reduced free operator path,
- the owned-row tangent machinery stays underneath the builder,
- the full-operator and BDDC-specific branches stay inactive.

This is the cleanest summary of what Newton is actually doing on the default parallel PMG route.

7.2 Plain Newton On The Mainline Path

One Newton iteration for the current benchmark is:

1. ask the constitutive builder for the current free-output residual and tangent information,
2. form the regularized operator

$$K_r = rK_{\text{elast}} + (1 - r)K_{\text{tangent}},$$

on the owned-row PETSc pattern,

3. extract or reuse the reduced free-space operator,
4. solve the correction with the PMG-shell-preconditioned Krylov solver,
5. run the line search,
6. update the state and regularization level.

The nonlinear method is still Newton. The main change is the operator format.

7.3 Nested SSR Solve On The Mainline Path

Because the current benchmark is an indirect SSR solve, the active Newton routine is `newton_ind_ssr(...)`. It still follows the classical MATLAB structure:

- build the V correction associated with the current residual,
- build the W sensitivity direction,
- recover d_λ from the work constraint,
- combine them into the displacement correction,
- damp the coupled update through `damping_alg5(...)`.

What differs from MATLAB is that both V and W are solved through the same reduced PMG-shell linear-system interface.

7.4 Stopping And Damping In The Current Benchmark

The committed benchmark config uses:

- `stopping_criterion=absolute_delta_lambda`,
- `stopping_tol=1e-4`,
- `it_damp_max=10`,
- `r_min=1e-4`.

So the mainline nonlinear solve is tuned around SSR progress in λ , not around a pure residual-only stopping rule.

7.5 What Newton Receives From The Constitutive Layer

On the mainline path, Newton primarily sees:

- free-output residual builders,
- owned-row tangent refreshes hidden behind the constitutive builder,
- cached PETSc regularized matrices on the owned pattern.

This is why the tangent and Newton chapters have to be read together. The current Newton branch is inseparable from the owned-row assembly path beneath it.

7.6 Deflation Basis On The Mainline Path

The solver-side deflation basis still grows after useful accepted directions, just as in MATLAB. On the current mainline path, that means:

- the basis is owned by the PETSc solver object,
- continuation and Newton can reuse it across accepted steps,
- the default benchmark does **not** enable temporary first-iteration warm-start modes.

7.7 Where To Edit

- `src/slope_stability/nonlinear/newton.py` for iteration flow and operator selection,

- `src/slope_stability/nonlinear/damping.py` for line-search behavior,
- `src/slope_stability/constitutive/problem.py` for the residual and regularized-matrix interfaces used by Newton,
- `src/slope_stability/linear/solver.py` for backend-specific expectations.

7.8 Where The Inactive Newton Branches Went

Full-system operator mode, local full-operator branches, separate explicit preconditioning matrices, and optional warm-start features are moved to [Appendix G](#).

Continuation Algorithms

The committed 3D benchmark follows one continuation path only:

SSR → indirect method → classic mode → secant predictor → legacy omega controller.

This chapter now follows only that path.

8.1 Mainline Continuation Contract

The benchmark defines or inherits the following effective continuation settings:

- `analysis=ssr`,
- `method=indirect`,
- `predictor=secant`,
- `omega_step_controller=legacy`,
- `continuation_mode=classic`,
- `continuation_secant_correction_mode=none`,
- `continuation_first_newton_warm_start_mode=none`,
- `step_length_cap_mode=none`,
- `fine_switch_mode=none`.

That list is the main reason the continuation chapter is cleaner than before: many repository features exist, but the default benchmark uses a fairly narrow subset of them.

8.2 Mainline Call Chain

The active continuation chain is:

1. `SSR_indirect_continuation(...)` in `src/slope_stability/continuation/indirect.py`,
2. initialization of the first accepted points,
3. a secant predictor for the next candidate state,
4. a nested Newton solve through `newton_ind_ssr(...)`,
5. acceptance or rejection of the candidate point,
6. update of continuation history and the solver deflation basis.

This is the modern PETSc equivalent of MATLAB `slope_stability_matlab/+CONTINUATION/SSR_indirect_continuation.m`.

8.3 Accepted-Step Lifecycle

One accepted continuation step on the current path is:

1. choose the next target ω ,
2. build the secant predictor $(U_{\text{ini}}, \lambda_{\text{ini}})$,
3. solve the nested Newton problem for that target,
4. accept the point if Newton succeeds,
5. append the accepted state to the branch history,
6. update the solver recycle basis.

Rejected attempts still exist, but the basic branch-tracking logic is the same one MATLAB authors already know.

8.4 What The Legacy Controller Means Here

Because the benchmark keeps `omega_step_controller=legacy`, the current default path uses the older, simpler continuation-step adaptation logic rather than the newer adaptive controller family. In practical terms:

- step growth and shrinkage are still driven by continuation success and Newton effort,
- there is no separate adaptive efficiency controller active,
- there is no fine-switch phase transition active.

8.5 Mainline History Output

The current path still records the accepted branch history familiar from MATLAB:

- λ history,
- ω history,
- displacement history,
- accepted-step and attempt statistics,
- linear and nonlinear work counters.

The difference is that PETSc stores these in explicit structured outputs rather than a mostly monolithic MATLAB struct.

8.6 Where To Edit

- `src/slope_stability/continuation/indirect.py` for the mainline SSR driver,
- `src/slope_stability/nonlinear/newton.py` for the nested Newton solve it calls,
- `src/slope_stability/core/run_config.py` if the mainline continuation controls should be exposed differently through TOML.

8.7 Where The Other Continuation Features Went

Direct SSR, limit load, adaptive controllers, reduced predictors, secant corrections, fine-switch logic, first-Newton warm starts, and streaming microsteps are moved to Appendix [H](#).

Part II

Annex: Inactive and Alternative Paths

Execution Map: Inactive and Alternative Entry Paths

This appendix chapter collects runner surfaces and configuration defaults that exist in the repository but are not part of the current `slope_stability_3D_hetero_SSR_default` benchmark path.

A.1 Generic Framework Defaults

If a user relied only on dataclass defaults in `src/slope_stability/core/run_config.py`, the framework-level defaults would be:

- `method=indirect`,
- `predictor=secant`,
- `omega_step_controller=legacy`,
- `constitutive_mode=overlap`,
- `tangent_kernel=rows`,
- `solver_type=PETSC_MATLAB_DFGMRES_HYPRE_NULLSPACE`,
- `pc_backend=hypre`.

That is not the same as the committed benchmark default. The committed 3D benchmark overrides the backend to `pmg_shell`.

A.2 Other Runner Surfaces

The public entrypoint for these flows is still `src/slope_stability/cli/run_case_from_config.py`. After config and asset resolution, `src/slope_stability/execution/asset_case/runner.py` dispatches to generic route modules that are not on the current mainline path:

- `src/slope_stability/execution/asset_case/seepage_3d.py` for standalone 3D seepage,
- `src/slope_stability/execution/asset_case/seepage_ssr_3d.py` for the coupled seepage-to-mechanics path,
- `src/slope_stability/execution/asset_case/mechanics_2d.py` and `src/slope_stability/execution/asset_case/seepage_2d.py` for 2D mechanics and seepage,
- the limit-load branch launched through `LL_indirect_continuation(...)`,
- direct SSR entry points exposed through `src/slope_stability/continuation/direct.py` and `src/slope_stability/continuation/omega.py`.

These are active code, but they are not what the default 3D PMG benchmark does when run from `case.toml`.

A.3 Why The Split Matters

Before this restructure, the document described both the mainline path and the broader feature surface together. That made it too easy to confuse:

- code that is *supported* with code that is *executed by default*,
- generic configuration defaults with benchmark overrides,
- validation/reference paths with production PMG-path assembly.

The main chapters now follow only the committed benchmark. This appendix holds the surrounding entry points so they stay documented without interrupting the main reading path.

Mesh, Materials, and Load Assembly: Inactive Paths

This appendix chapter collects preprocessing routes that remain in the codebase but are not part of the default parallel PMG benchmark path.

B.1 Global Reference Assembly

The repository still contains the more MATLAB-like route:

1. `assemble_strain_operator(...)`,
2. `build_elastic_stiffness_matrix(...)`,
3. `vector_volume(...)`.

That path builds the global **B** matrix, global **WEIGHT**, global **K_{elast}**, and global **f_V** exactly in the MATLAB style. It remains important for validation, serial comparison, and direct formula tracing, but it is not the normal PMG startup path.

B.2 Asset-First Mesh Loading

The active benchmark path uses canonical problem assets:

- `meshes/<asset>/definition.py` declares mesh variants, materials, mechanics BCs, seepage BCs, hydraulic constants, and profiles,
- `src/slope_stability/assets/support/canonical_gmsh.py` reads canonical linear Gmsh meshes and promotes them to the requested element order,
- `src/slope_stability/problem_asset_runtime.py` resolves the selected `asset`, `mesh_variant`, and optional `profile`.

Older water-level and COMSOL-specific mesh loaders are historical migration references only; seepage and coupled workflows now enter through the same asset API as dry mechanics cases.

B.3 Alternative Node Orderings

`src/slope_stability/mesh/reorder.py` still supports orderings beyond `blockmetis`:

- `xyz`,
- `morton`,
- `rcm`.

Those orderings remain useful for experiments and diagnostics, but the committed 3D PMG benchmark is written around `block_metis`.

B.4 Why The Global Path Remains Important

Although the owned-row path is mainline, the global path still serves three purposes:

- it is the shortest conceptual bridge back to MATLAB,
- it is the easiest correctness reference for distributed assembly,
- it is the cleanest serial debugging path.

Seepage Coupling: Inactive Branch

This appendix chapter covers the hydro-mechanical branch used by seepage variants, especially `slope_stability_matlab/slope_stability_3D_hetero_seepage_SSR.m`. It is not active in the committed default 3D PMG benchmark, but it is still a supported branch in the repository.

C.1 Exact Seepage-to-Mechanics Sequence

The preserved numerical sequence is:

```
conduct0 → boundary conditions → seepage solve
        → (pw, grad_p, mater_sat)
        → mechanical  $\gamma$  and f_V.
```

Stage	MATLAB location	PETSc location
Mesh, labels, materials, and head BCs	+MESH/load_mesh_gmsh_waterlevels and related helpers	<code>meshes/<asset>/definition.py</code> , <code>src/slope_stability/assets/support/canonical_gmsh.py</code>
Conductivity build	+SEEPAGE/heter_conduct.m	<code>src/slope_stability/seepage/flow.py</code>
Boundary conditions	seepage boundary helpers under +MESH	asset-owned head BCs resolved by <code>src/slope_stability/problem_asset_runtime.py</code>
Seepage Newton solve	+SEEPAGE/seepage_problem_3D.m, <code>newton_flow.m</code>	<code>src/slope_stability/seepage/flow.py</code>
Transfer into mechanics	script-local coupling logic	<code>src/slope_stability/execution/asset_case/seepage_ssr_3d.py</code>

C.2 What The PETSc Split Changed

The seepage branch is no longer embedded inside one large driver script. PETSc separates:

- asset resolution and canonical mesh loading,
- seepage assembly and Newton flow,
- transfer of `pw`, `grad_p`, and saturation into the mechanical stage,
- the final SSR mechanical solve.

The mathematics is unchanged. The packaging is modular so seepage can be run and debugged independently.

C.3 Main Edit Points

- `src/slope_stability/seepage/flow.py` for seepage numerics,
- `meshes/<asset>/definition.py` for seepage geometry ownership, materials, conductivity, water unit weight, and head conditions,
- `src/slope_stability/execution/asset_case/seepage_3d.py` for standalone seepage,
- `src/slope_stability/execution/asset_case/seepage_ssr_3d.py` for the coupled branch.

Constitutive Relationships and Reduction: Inactive Modes

This appendix chapter collects constitutive execution modes and builder interfaces that are present in the repository but inactive in the committed parallel PMG benchmark.

D.1 Alternative Ownership Modes

Mode	Meaning	Why it exists
<code>global</code>	build and use global B, S, and DS arrays	closest MATLAB parity and easiest validation path
<code>unique / unique_gather</code>	evaluate only unique integration points, then gather	explicit ownership semantics
<code>unique_exchange</code>	unique-owner evaluation plus neighbor exchange	lower communication than full gather

These are execution strategies, not different constitutive laws.

D.2 Why They Are Not Mainline Today

For the committed 3D benchmark:

- `global` is a reference and validation path,
- `unique*` modes are alternative MPI ownership experiments,
- the default run stays in `overlap` because it maps directly onto the owned-row tangent assembly used by the PMG path.

D.3 Additional Builder Interfaces

The Python constitutive layer also exposes interfaces that are not central to the default PMG run:

- `build_F(...)` and `build_F_all(...)` for full-field outputs,
- `build_F_local(...)` and `build_F_free_local(...)` for local full-operator branches,
- `build_bddc_*` methods for BDDC local-subdomain matrices.

They are documented here because they matter for solver backends other than the mainline `pmg_shell` route.

Tangent Assembly: Inactive and Alternative Paths

This appendix chapter collects tangent paths that remain in the repository but are not part of the default parallel PMG benchmark.

E.1 Global MATLAB-Style Reference Path

`build_global_tangent_matrix(...)` still assembles the full global matrix directly as

$$K = B^T D_S B.$$

This is the shortest conceptual bridge to MATLAB and remains valuable for serial debugging and parity checks, but it is not the normal PMG-path solve.

E.2 Unique-Owner Constitutive Recovery

The owned tangent pattern can also be prepared with unique-owner support:

- `include_unique=true`,
- optional gather or neighbor exchange metadata,
- mapping arrays that recover overlap-local constitutive data from unique owners.

Those structures matter for `unique`, `unique_gather`, and `unique_exchange` modes. They are inactive in the committed benchmark because `constitutive_mode=overlap`.

E.3 Legacy Scatter Kernel

The `legacy` kernel is the closer relative of the MATLAB assembly mental model:

- build an element matrix,
- flatten it,
- scatter entries through a precomputed map.

It is still useful as a reference and validation kernel, but it is not the default production kernel for the 3D PMG benchmark.

E.4 BDDC Local-Subdomain Pattern

`BDDCSubdomainPattern` serves a different PETSc consumer: local square subdomain matrices for MA-

TIS/PCBDDC. Its main differences from the owned-row pattern are:

- local square numbering instead of global owned rows,
- explicit interface and adjacency metadata,
- tangent refresh routines specialized for the BDDC preconditioner.

This branch becomes relevant only when the solver backend is changed to `bddc`.

E.5 Why The Main Text Excludes Them

The mainline chapter now follows one concrete path:

owned rows $\rightarrow$ overlap constitutive data $\rightarrow$ rows kernel.

The inactive paths are kept here so they remain documented without obscuring which branch the default benchmark actually executes.

Linear Solvers and PETSc: Inactive Backends and Policies

This appendix chapter collects solver families, preconditioner backends, and policy knobs that exist in the repository but are not part of the committed parallel PMG benchmark.

F.1 Other Solver Families

The factory in `src/slope_stability/linear/solver.py` also supports:

- direct solvers,
- pure Python deflated FGMRES,
- PETSc Krylov solves without the MATLAB-style wrapper,
- experimental PETSc-native deflation variants.

These are useful for debugging, comparison, or research, but not the default benchmark path.

F.2 Other Backends

The same outer solver family can run with several backends:

- `hypre`,
- `gamg`,
- `bddc`,
- built-in PETSc `pmg`,
- LU/direct factorization.

Only `pmg_shell` is mainline in the committed 3D benchmark.

F.3 Why BDDC Is Separate

The BDDC backend changes more than a PETSc option. It changes the matrix shape needed by the preconditioner, because it wants the local subdomain operators built through `BDDCSubdomainPattern`. That is why BDDC is kept out of the mainline text.

F.4 Other Explicit Policy Controls

The broader solver framework exposes policies that the PMG-shell mainline path does not vary:

- `preconditioner_matrix_source` other than `tangent`,
- lagged or accepted-step matrix policies,
- rebuild intervals other than every Newton iteration,
- full-system preconditioning modes.

Those options remain relevant for alternative backends and experiments.

Newton Solvers and Damping: Inactive Branches

This appendix chapter collects Newton interfaces and solver interactions that remain implemented but are not exercised by the committed default PMG benchmark.

G.1 Full-System Operator Mode

Some solver backends prefer the full operator instead of the reduced free-space matrix. In those cases Newton can keep a full PETSc operator and let the solver manage free indices internally. This branch is inactive for `pmg_shell` because `full_system_preconditioner=false`.

G.2 Local Full-Operator Builders

The constitutive layer also exposes:

- `build_F_local(...)`,
- `build_F_free_local(...)`,
- related local-output interfaces.

These support PETSc full-operator workflows that consume owned-row local data directly. They are not the mainline PMG-shell path.

G.3 Separate Explicit Preconditioning Matrices

The broader PETSc framework can request an explicit preconditioning matrix separate from the operator used in the Krylov solve. Depending on solver policy, that matrix can come from:

- the elastic operator,
- the tangent operator,
- the regularized operator.

The default PMG-shell benchmark keeps a narrower, fixed policy and therefore does not need the full generality discussed above.

G.4 Optional Warm Starts

The code also supports temporary first-iteration warm-start bases passed from continuation into Newton. Those features remain inactive in the committed benchmark because `continuation_first_newton_warm_start_mode=none`.

Continuation Algorithms: Inactive Modes and Extensions

This appendix chapter collects continuation branches and control-flow extensions that remain in the repository but are not active in the committed default PMG benchmark.

H.1 Direct SSR

The direct SSR path is still implemented through:

- `src/slope_stability/continuation/direct.py`,
- `src/slope_stability/continuation/omega.py`.

It remains the closest PETSc analogue of the simpler MATLAB direct continuation scripts, but it is not the current default path.

H.2 Limit Load

The limit-load branch remains available through `src/slope_stability/continuation/limit_load.py`. It solves a different continuation problem with the load multiplier t instead of the SSR reduction factor λ . This is active code, but outside the current benchmark.

H.3 Predictor Families Beyond Secant

`src/slope_stability/continuation/indirect.py` supports predictor families beyond the mainline secant mode:

- `reduced_newton_all_prev`,
- `reduced_newton_affine_all_prev`,
- `reduced_newton_window`,
- `reduced_newton_increment_power`.

These change the quality of the initial guess, not the continuation equations.

H.4 Additional Control-Flow Extensions

The inactive feature set also includes:

- adaptive omega-step control,

- secant correction modes,
- history-box step-length caps,
- fine-switch logic,
- temporary first-Newton warm starts,
- `streaming_microstep` continuation mode.

These are real PETSc-era extensions, but they are not part of the committed default 3D benchmark configuration.

H.5 Why They Were Moved Out Of The Main Text

The mainline continuation chapter now follows the exact benchmark settings: `indirect+secant+legacy+classic`. The alternatives remain documented here so the document stays useful for future feature work without obscuring what the default benchmark actually does today.